COSC 3430 · ARTIFICIAL INTELLIGENCE

End-to-End Machine Learning Project

Part 2: get the data, and take a first look

Based on Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Ch. 2

ROADMAP

Ten steps, over several code-along lectures

1. Look at the big picture
2. Get the data
3. Explore and visualize the data
4. Create the test data
5. Prepare the data for learning
6. Select a model and train it
7. Fine-tune the model
8. Present your solution
9. Launch the production system
10. Monitor and maintain it

Today: step 2, and a first look at step 3.

See also: Géron's ML project checklist (github.com/ageron/handson-mlp)

STEP 2 · CHECKLIST

Get the data, and automate it

1 List the data you need, and how much
2 Find and document where to get it
3 Check how much space it takes
4 Check legal obligations; get authorization
5 Get access authorizations
6 Create a workspace with enough storage
7 Get the data
8 Convert it to an easy format (without changing the data)
9 Delete or protect sensitive information
10 Check size and type (time series, sample, geographical ...)
11 Set aside a test set and never look at it: no data snooping!

Automate as much as possible, so you can easily get fresh data

→ SWITCH TO EDITOR · CODE-ALONG 2

Fetch the data

1. Unpack the compressed tarball **housing.tgz** with Python's **tarfile**
2. List the result: one file, **housing.csv**, about 1.4 MB
3. Load it into a pandas **DataFrame** straight from the web

tinyurl.com/housing-csv

```
housing_full = pd.read_csv(…)
```

→ SWITCH TO EDITOR · CODE-ALONG 3

Get a quick overview

1. **head()**: the first five rows, one district per row
2. **info()**: rows, columns, types, non-null counts
3. **value_counts()** on the one non-numeric column
4. **describe()**: summary statistics of the numeric columns

Pause after info() and after describe(): questions follow

QUESTION

What does info() tell you about the data?

ANSWER

20,640 districts, 10 columns, one gap

- 10 columns: **9 numeric** (float64), **1 object**
- 20,640 rows, one per district
- **total_bedrooms** has 207 missing values (20,433 non-null)
- **ocean_proximity** is an object; read from a CSV, it must be text

ocean_proximity	count
<1H OCEAN	9,136
INLAND	6,551
NEAR OCEAN	2,658
NEAR BAY	2,290
ISLAND	5

<1H OCEAN = less than one hour from the ocean

QUESTION

What do we learn from describe()?

ANSWER

Center, spread and skew in a few numbers

median_house_value	USD
mean	206,856
std	115,396
25%	119,600
50% (median)	179,700
75%	264,725
max	500,001

- Missing values (NaN) are ignored
- **mean > median**: a long right tail pulls the mean up
- **std** measures spread around the mean
- **IQR** = 75% − 25% = **145,125**: the middle half of the districts
- max = 500,001: suspicious. Keep an eye on it.

Next: make the same numbers visible

→ SWITCH TO EDITOR · CODE-ALONG 4

A box plot of the target

1. Import **matplotlib.pyplot**
2. Box plot of **median_house_value**, in portrait format
3. Add a title and a y-axis label (USD); save the figure
4. The box is the IQR; the line inside is the median

```
housing_full.boxplot(column="median_house_value", figsize=(4, 6))
```

QUESTION

What do you notice at the top of the box plot?

ANSWER

The top is capped at $500,001

Many districts pile up at exactly the same maximum value. That's a strong artificial pattern, and we don't want the model to learn it.

Capping (clipping)

Values above a threshold are **kept** but recorded as the threshold: a $1,000,000 district shows up as $500,001.

Truncating

Values beyond the threshold are **removed** from the data altogether.

Capping at a percentile is also called winsorizing

→ SWITCH TO EDITOR · CODE-ALONG 5

Histograms of every numeric feature

1. Clear the previous figure
2. One call draws a histogram per numeric column: **hist()**
3. Use 50 bins and a landscape figure; save it
4. Look at every panel before the next question

```
housing_full.hist(bins=50, figsize=(12, 8))
```

QUESTION

What do you see in the histograms? Share all your observations.

ANSWER

Six observations

1 · Capped

median_house_value (500,001) and housing_median_age (52) spike at their maximum

2 · Not in USD

median_income is in tens of thousands of dollars (3 ≈ $30,000), clipped at 0.5 and 15

3 · Right-skewed

total_rooms, total_bedrooms, population, households: long right tails pull the mean above the median

4 · Different scales

From single digits to tens of thousands: features need scaling before training

5 · Two peaks

latitude and longitude are bimodal: the Bay Area and Los Angeles, as on the Part 1 map

6 · Key predictor

median_income matters most for price: we'll stratify the test set by it

FOLLOW-UP QUESTION

Which features would you feed straight into a linear regression, and which would you transform first? Why?

ANSWER

Transform the skewed ones; watch the target

Treatment	Features	Why
Log-transform	total_rooms, total_bedrooms, population, households	heavily right-skewed
Use, with care	median_income, housing_median_age	roughly usable, but clipped or capped
Not linear in price	latitude, longitude	two metro peaks; better as engineered features
Problem target	median_house_value	capped: no prediction above $500,001

Why: linear regression assumes a straight-line relation between each feature and the target. Skew squeezes most values into a narrow range; a log transform spreads them out.

We handle these when we prepare the data for training

WRAP-UP

What we know about the data so far

- Supervised, batch, multiple univariate regression, scored with RMSE
- 20,640 districts, 10 columns, 207 missing bedroom counts
- Capped target, skewed features, very different scales

Next: set aside a test set before we look any closer.